

Comprehensive HR System Architecture on Supabase — No Payroll Engine

Executive summary

Your vision is well suited to a **Supabase-centered HR information system** in which Supabase provides almost the entire backend: PostgreSQL for HR data, Auth for identity, Row Level Security for authorization, Storage for employee documents, Realtime for live portal updates, Edge Functions for privileged workflows and integrations, Cron for scheduled processes, Queues for reliable background jobs, and Vault/project secrets for credentials. Supabase is specifically designed so authenticated applications can query PostgreSQL directly while RLS determines which rows each user is permitted to access. ¹

The most important architectural decision is to **not build four separate applications** for Employee, Supervisor, Manager, and Admin. Build one HR platform with one identity system and a permission/scope model. The frontend presents a different portal experience depending on the person's assignments, but PostgreSQL RLS remains the authoritative security boundary. Supabase supports custom claims/RBAC, while RLS can express row-level rules directly in PostgreSQL. ²

The proposed product would cover:

Area	Recommended capability
Employee records	Profiles, employment history, position, department, supervisor, manager, status, contacts, custom fields
Attendance	Clock-in, clock-out, breaks, schedules, late/early flags, corrections, optional geofencing
Leave	Employer-defined leave types, requests, multi-stage approvals, balances, accruals, attachments
Onboarding	Employer-built templates, steps, dependencies, forms, documents, acknowledgements, training and approvals
Performance	Employer-defined appraisal templates, appraisal cycles, recurring checkpoints, self-assessments, manager reviews, approvals
Organization	Departments, teams, locations, positions, reporting hierarchy and historical assignments
Documents	Contracts, policies, certificates, IDs, employee documents, versioning and acknowledgements
Notifications	In-app notifications plus external email/SMS/push where required

Area	Recommended capability
Reporting	Attendance, absenteeism, leave, headcount, onboarding, appraisal, organizational and payroll-import reporting
Audit	Application audit events, Supabase Auth audit events, administrative actions and sensitive-data access
Payroll interface	Import externally calculated payroll from Excel/XLSX; never calculate payroll internally
Security	RLS, scoped RBAC, SSO, MFA, private Storage, audit trails, separation of sensitive data
Additional HR modules	Offboarding, training/certifications, equipment/assets, employee changes, emergency contacts and configurable forms

The payroll distinction should remain explicit throughout the product:

The HR system stores and reconciles payroll results. It does not calculate payroll.

An administrator uploads the Excel workbook produced by the employer's external payroll process. The system validates the file, matches each row to an employee, reports unmapped or ambiguous rows, and creates immutable payroll-import records. A correction is imported as a revision rather than silently overwriting historical payroll information. This preserves auditability while still allowing the newest imported information to automatically become the current payroll record shown in the system.

For normal application CRUD operations, you do **not** need to create a conventional custom REST backend. Supabase exposes the database through its Data API and client libraries, with the authenticated user's JWT participating in RLS enforcement. Edge Functions should be reserved for operations that require elevated credentials, transactional business workflows, spreadsheet processing, user invitations, external notifications, e-signature integrations and similar privileged actions. ³

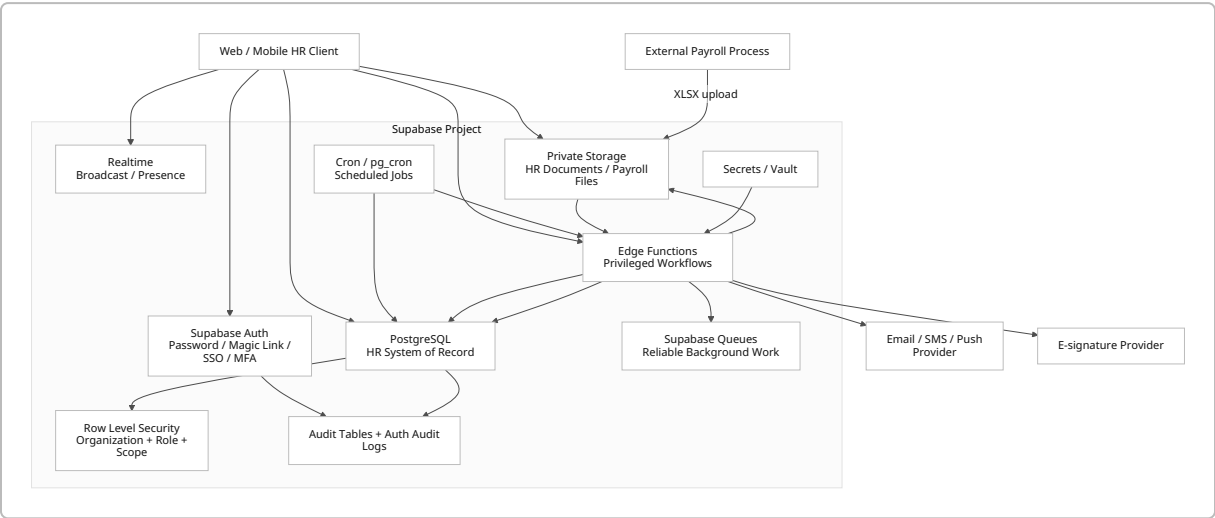
Architecture and foundational design

I recommend making the architecture **organization-aware from day one**, even if the first customer is a single employer. Every HR-domain row should normally contain an `organization_id`. That allows the same codebase eventually to support multiple employers without redesigning every table and RLS policy.

An employee should also be separated conceptually from an authenticated user:

- `auth.users` represents the login identity.
- `employees` represents the HR/employment record.
- `employees.user_id` can initially be `NULL`, allowing HR to create a pre-hire employee before the person receives an account.
- When invited, the employee record is linked to the appropriate `auth.users.id`.

Supabase itself recommends creating application-domain profile tables referencing `auth.users` rather than treating the Auth schema as your application's user database. ⁴



This architecture fits Supabase's intended model: PostgreSQL forms the underlying database, Auth integrates with JWT/RLS, Storage uses RLS-based access controls, Realtime provides Broadcast/Presence/database-change features, and Edge Functions provide server-side TypeScript for integrations and privileged operations. ⁵

Supabase service map

Supabase capability	HR use	Required?	Important implementation note
PostgreSQL	HR records, leave, attendance, appraisals, payroll imports, audit data	Yes	Primary system of record.
Auth	Login, passwordless, OAuth, SSO, MFA	Yes	Supabase Auth supports passwords, passwordless methods, social providers and SSO. ⁶
RLS	Employee/manager/admin authorization	Yes	Enable it on every exposed application table; grants and RLS are separate security layers. ⁷
Custom Access Token Hook	Optional role/organization claims in JWT	Recommended	Supabase supports adding claims before token issuance. ⁸
Realtime	Dashboard updates, approval changes, live clock status	Recommended	Private Broadcast/Presence channels can themselves be protected by RLS on <code>realtime.messages</code> . ⁹

Supabase capability	HR use	Required?	Important implementation note
Storage	Contracts, sick-note attachments, appraisal attachments, uploaded XLSX files	Yes	Keep HR buckets private; private-bucket download operations remain subject to authorization/RLS. ¹⁰
Edge Functions	Invitations, Excel processing, external APIs, sensitive workflows	Yes	Functions run server-side TypeScript and can securely use backend credentials. ¹¹
Cron	Appraisal generation, reminders, leave accrual, overdue checks	Recommended	Supabase Cron uses <code>pg_cron</code> ; jobs can execute SQL/functions or invoke Edge Functions. ¹²
Queues	Notification delivery, large import orchestration, retries	Recommended	Supabase Queues provides durable PostgreSQL-native message queues. ¹³
Vault / secrets	External API credentials	Recommended	Vault provides encrypted secret storage in PostgreSQL; Edge Functions also support project secrets. ¹⁴
Auth Audit Logs	Login/password/token security events	Yes	Auth audit logs automatically capture authentication events. ¹⁵
Platform Audit Logs	Supabase administrative/project changes	Optional	Currently a Team/Enterprise capability rather than your application's HR audit trail. ¹⁶

Authentication strategy

Start with employer-controlled invitations rather than public signup.

An Admin creates the employee first, then invokes an `invite-employee` Edge Function. That function performs the privileged Auth administration operation, links the resulting Auth user to `employees.user_id`, records the invitation in the application audit trail and sends or triggers the appropriate invitation.

Administrative Auth operations must stay server-side because Supabase secret/service-role credentials bypass RLS and must never be exposed in browser or mobile code. ¹⁷

Authentication options should include:

Standard organizations: work email + password or email OTP/magic link. Supabase provides email passwordless login using Magic Links or OTPs. ¹⁸

19

20

21

Feature	Recommended tables and important fields	Design notes
Positions	<code>positions(id, organization_id, title, job_code, description, grade, active)</code>	Position is the role in the company, not the application's security role.
Assignments	<code>employee_assignments(id, employee_id, org_unit_id, position_id, supervisor_employee_id, manager_employee_id, location_id, start_date, end_date, is_primary)</code>	Preserves job/department/reporting history rather than overwriting it.
Security roles	<code>role_assignments(id, user_id, organization_id, role, scope_type, scope_id, valid_from, valid_until);</code> optionally <code>role_permissions</code>	Supports one person being Employee + Supervisor + Manager simultaneously.
Attendance	<code>attendance_sessions(id, organization_id, employee_id, work_date, clock_in_at, clock_out_at, status);</code> <code>attendance_events(id, session_id, event_type, occurred_at, source, device_metadata, location);</code> <code>attendance_adjustments</code>	Keep event history even after an HR correction.
Schedules	<code>work_schedules</code> , <code>schedule_assignments</code> , <code>schedule_shifts</code> , <code>holidays</code> , <code>attendance_policies</code>	Allows lateness, overtime display and expected working-time calculations without becoming payroll.
Leave	<code>leave_types</code> , <code>leave_policies</code> , <code>leave_requests</code> , <code>leave_request_days</code> , <code>leave_approvals</code> , <code>leave_ledger</code>	Make leave types employer-configurable rather than hard-coding sick/vacation.
Onboarding	<code>onboarding_templates</code> , <code>onboarding_template_versions</code> , <code>onboarding_template_steps</code> , <code>onboarding_runs</code> , <code>onboarding_tasks</code>	Version templates; an active employee onboarding run should retain the version that existed when it began.
Appraisals	<code>appraisal_templates</code> , <code>appraisal_sections</code> , <code>appraisal_questions</code> , <code>appraisal_cycles</code> , <code>appraisal_instances</code> , <code>appraisal_reviewers</code> , <code>appraisal_responses</code> , <code>appraisal_signoffs</code>	A cycle can create recurring or one-time checkpoints.

Feature	Recommended tables and important fields	Design notes
Documents	<code>documents</code> , <code>document_versions</code> , <code>document_acknowledgements</code> , <code>signature_requests</code>	Store file metadata in PostgreSQL and binaries in private Supabase Storage.
Payroll import	<code>payroll_import_batches</code> , <code>payroll_import_rows</code> , <code>payroll_column_maps</code> , <code>payroll_import_errors</code>	External payroll results only. Never calculate taxes/wages/ deductions.
Notifications	<code>notifications</code> , <code>notification_preferences</code> , <code>notification_delivery_attempts</code>	PostgreSQL stores notification state; Realtime makes the UI update immediately.
Reporting	PostgreSQL views such as <code>attendance_summary_v</code> , <code>leave_balance_v</code> , <code>headcount_v</code> , <code>appraisal_progress_v</code>	Create exposed reporting views using <code>security_invoker=true</code> so underlying RLS remains effective. Supabase warns that conventional views may otherwise bypass RLS. ²²
Audit	<code>audit_events(id, organization_id, actor_user_id, employee_id, action, entity_type, entity_id, old_data, new_data, request_id, created_at)</code>	Business audit trail is separate from Supabase's Auth/platform logging.
Offboarding	<code>offboarding_templates</code> , <code>offboarding_runs</code> , <code>offboarding_tasks</code>	Reuse the workflow engine concepts used for onboarding.
Learning/ compliance	<code>training_courses</code> , <code>employee_training</code> , <code>certifications</code>	Useful for mandatory certification expiry/reminders.
Equipment/ assets	<code>assets</code> , <code>employee_asset_assignments</code>	Supports laptop, phone, badge and equipment return during offboarding.
Custom employer fields	<code>custom_field_definitions</code> , <code>employee_custom_field_values</code>	Avoid modifying the schema every time one employer needs an additional HR field.

Employee management

The Employee module should become the center of the product, but the `employees` record should not contain every HR fact.

The employee page should aggregate:

Identity: name, preferred name, employee number, work email and account status.

Employment: hire date, employment status, current position, department, work location, supervisor and manager.

History: previous positions, department changes and reporting changes from `employee_assignments`.

Time: recent attendance, lateness/corrections and leave.

Performance: active checkpoints, previous appraisals and goals.

Workflow: onboarding/offboarding progress.

Documents: accessible employee documents.

Training: completed and expiring certifications.

Payroll-import history: only where the user has explicit payroll-view permission.

The employee record should therefore behave like a **composed profile**, not a single database row.

Attendance sign-in and sign-out

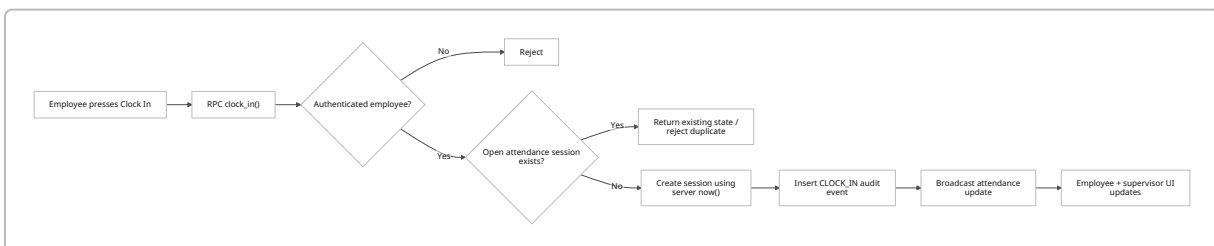
Attendance must be persistent database state. **Do not treat Supabase Presence as the attendance system of record.** Presence is designed for connected-client state and changes when clients connect, disconnect or update their Presence payload. ²³

Use a server timestamp for the authoritative clock event.

Clock-in:

1. Employee taps **Clock In**.
2. Frontend calls `rpc('clock_in', metadata)` rather than inserting directly into the table.
3. PostgreSQL verifies the authenticated employee, checks that an open session does not already exist and records `now()`.
4. A corresponding immutable `attendance_event` is inserted.
5. Realtime notifies the employee portal and authorized supervisor dashboard.

Clock-out does the corresponding operation.



Create a partial unique index to enforce only one open attendance session per employee:


```
create unique index attendance_one_open_session
on public.attendance_sessions(employee_id)
where clock_out_at is null;
```

That database constraint is far stronger than relying on the frontend button being disabled.

Optional attendance additions should include break start/end, shift schedules, overnight shifts, manual adjustment requests, supervisor corrections, kiosk mode, work-from-home indicator, optional location capture and missed-clock alerts.

For optional geofencing, store office coordinates and evaluate the submitted location at the clock event. Do not continuously track employee location unless the employer actually has a justified requirement.

Realtime is useful here for dashboards. Supabase supports private Broadcast/Presence authorization through RLS policies on `realtime.messages`. ²⁴

Leave and "all types of days"

Do not model:

```
sick_days
vacation_days
personal_days
```

as separate table structures.

Instead, create an employer-controlled `leave_types` table:

```
Annual Vacation
Sick Leave
Personal Leave
Bereavement
Parental Leave
Unpaid Leave
Study Leave
Jury Duty
Medical Appointment
Compassionate Leave
Remote Work
Other
```

Each employer can configure:

```
requires_approval
requires_attachment
attachment_after_days
paid_indicator
balance_tracked
allow_half_day
allow_hourly
allow_negative_balance
minimum_notice_days
maximum_consecutive_days
approval_workflow
```

Use a **leave ledger** rather than relying solely on a mutable numeric balance:

```
+15.00  annual grant
-2.00   approved vacation
+1.25   adjustment
+1.00   monthly accrual
-0.50   half-day leave
```

`leave_balance_v` can sum ledger transactions. This creates a much clearer audit history than repeatedly modifying `balance = balance - x`.

Cron can run monthly or annual leave accruals, carryover operations and expiry processes. Supabase Cron can invoke SQL/database functions directly or make HTTP calls to an Edge Function. ²⁵

Employer-configured onboarding

The employer should build workflows rather than developers coding one fixed onboarding sequence.

Example:

```
New Employee Onboarding v4
|
├─ Complete employee information
├─ Upload identification
├─ Read employee handbook
├─ Sign confidentiality agreement
├─ Manager confirms workstation
├─ IT assigns laptop
├─ Complete safety training
└─ Supervisor first-week meeting
```

- └─ 30-day checkpoint
- └─ HR final verification

Each `onboarding_template_step` should include approximately:

```
id
template_version_id
title
description
step_type
assignee_type
sequence
due_offset_days
required
dependency_step_ids
form_schema_json
conditional_rules_json
document_template_id
requires_signature
```

`step_type` can support:

```
TASK
FORM
DOCUMENT_UPLOAD
DOCUMENT_REVIEW
ACKNOWLEDGEMENT
SIGNATURE
TRAINING
MEETING
APPROVAL
CHECKPOINT
```

When an employee starts, instantiate an `onboarding_run` and create concrete `onboarding_tasks`. Do not dynamically read the latest template after the process has started because editing the employer's future onboarding template should not unexpectedly rewrite an employee's historical process.

Cron should scan unfinished tasks for approaching due dates and overdue status. Notifications are inserted into `notifications`, and external email delivery can be queued if configured.

Appraisals and checkpoints

Your idea of an appraisal being a configurable **checkpoint** is stronger than limiting the system to "annual appraisal."

The employer should be able to create:

```
Probation – 30 days
Probation – 60 days
Probation – 90 days
Quarterly performance checkpoint
Mid-year review
Annual appraisal
Promotion assessment
Performance improvement checkpoint
Ad-hoc manager review
```

An `appraisal_template` defines the questionnaire and process. An `appraisal_cycle` defines **when and for whom** it runs.

Example cycle:

```
Template: Annual Performance Review 2026
Employee group: All active full-time employees
Start: November 1
Self-review due: November 14
Supervisor review due: November 28
Manager moderation due: December 5
Employee acknowledgement due: December 12
Repeat: annually
```

Questions should support text, numeric rating, rating scale, yes/no, multiple choice and goal/KPI evaluation.

Keep workflow status explicit:

```
DRAFT
OPEN
SELF_REVIEW_SUBMITTED
SUPERVISOR_REVIEW
MANAGER_REVIEW
HR_REVIEW
EMPLOYEE_ACKNOWLEDGEMENT
COMPLETE
CANCELLED
```

Cron creates appraisal instances when a cycle opens, marks late steps and generates reminders. This maps directly to Supabase's scheduled SQL/Edge Function capabilities. ²⁶

A powerful extension is a general `checkpoints` layer used by onboarding, probation and ongoing performance. That makes "90-day probation review" and "quarterly appraisal" variations of the same fundamental workflow concept.

Documents and e-signatures

Supabase Storage is appropriate for employee documents and supports fine-grained access controls through RLS. Private buckets are private by default, and downloads from private buckets require an authorized request or temporary signed URL. ²⁷

Recommended private buckets:

```
employee-documents
leave-attachments
appraisal-attachments
payroll-imports
company-policies
signature-artifacts
```

Do not put all objects in one unstructured folder.

Example:

```
employee-documents/
  {organization_id}/
    {employee_id}/
      contract/
      identification/
      certificates/
```

Your PostgreSQL `documents` table owns metadata and authorization; Storage owns the binary.

For simple HR acknowledgements such as "I acknowledge receiving the employee handbook," the system can implement this internally:

```
employee_id
document_version_id
acknowledged_at
ip_address
user_agent
auth_user_id
document_hash
```

For legally significant signatures where identity verification, tamper evidence, certificates and jurisdiction-specific signature requirements matter, use an external e-signature service through an Edge Function rather than attempting to reproduce an entire signature trust platform yourself.

Notifications

Use two layers.

In-app notifications: completely Supabase-native. Insert a `notifications` row and use Realtime so a connected client receives the update immediately. Database Broadcast can send authenticated private-channel messages. ²⁸

Email/SMS/push: requires a delivery channel. Supabase Edge Functions are explicitly intended to integrate with third-party systems and can send transactional email through external services. Supabase's built-in/default Auth SMTP is intended primarily for development and authentication flows; Supabase recommends configuring a production SMTP provider for production Auth email. ²⁹

Notification event examples:

```
leave.requested
leave.approved
leave.rejected
leave.cancelled
onboarding.task_assigned
onboarding.task_due
appraisal.opened
appraisal.review_due
attendance.missing_clock_out
document.expiring
training.expiring
employee.invited
payroll.import_completed
```

Store event creation separately from delivery status so temporary email-provider failure does not lose the notification.

Portals, RBAC and RLS authorization

The four portal concepts should be retained, but permissions should be more granular than four hard-coded account types.

A Supervisor can also be an Employee. A Manager is usually also an Employee. Some Admins should be allowed to manage employee records but perhaps **not payroll imports**. Therefore security should use roles plus explicit permissions and scope.

Recommended broad roles:

```
employee
supervisor
manager
admin
```

Recommended permissions underneath them:

```
employee.read_self
employee.read_team
employee.read_org
employee.update_self

attendance.clock_self
attendance.read_team
attendance.adjust_team

leave.request_self
leave.approve_direct_reports
leave.approve_unit

appraisal.complete_self
appraisal.review_direct_reports
appraisal.manage_cycles

onboarding.complete_self
onboarding.manage_team
onboarding.manage_templates

documents.read_self
documents.manage_team
documents.manage_org

payroll.read_self
payroll.import
payroll.read_org

reports.team
reports.org

roles.manage
```

This allows an employer eventually to customize role bundles without changing application code.

Portal behavior

Portal	Typical UI	Default data scope
Employee	My profile, clock in/out, attendance, leave, onboarding tasks, appraisals, documents, notifications, optional imported payslips	Self
Supervisor	Employee portal + direct reports, team attendance, leave approvals, assigned appraisal reviews, onboarding tasks	Direct reports
Manager	Employee portal + broader organizational dashboards, subordinate teams, escalated approvals, department reporting	Scoped org-unit subtree
Admin	Employee administration, org structure, workflow configuration, leave policies, appraisal templates, reports, imports, audit	Organization-wide according to permissions

The frontend can use role claims to determine navigation, but **the navigation is not security**. RLS must independently enforce the same restriction.

Supabase's RLS model allows policies to combine authenticated identity with SQL predicates, and Supabase's RBAC guide uses custom Access Token claims for role authorization. [30](#)

Recommended RLS matrix

Resource	Employee	Supervisor	Manager	Admin
Employee directory	Self + permitted public directory fields	Self + direct reports	Assigned org subtree	Organization
Private employee data	Self	Usually no	Usually no	Explicit HR permission
Attendance	Own	Direct reports	Scoped subtree	Organization
Leave request	Own	Direct-report requests	Scoped requests	Organization
Leave balance	Own	Limited direct-report visibility	Limited scoped visibility	HR permission
Appraisal	Own applicable sections	Assigned reviews	Assigned/ scoped reviews	Performance-admin permission

Resource	Employee	Supervisor	Manager	Admin
Onboarding	Own tasks	Assigned team tasks	Scoped progress	Admin
Documents	Own permitted documents	Explicitly shared	Explicitly shared	Permission based
Payroll import rows	Own final record only if enabled	No by default	No by default	<code>payroll.read_org</code>
Payroll import batch	No	No	No	<code>payroll.import</code>
Audit events	Own limited history at most	No	Limited	<code>audit.read</code>
Role assignments	Read own role	No	No	<code>roles.manage</code>

A critical principle is that **Manager should not automatically equal HR**. A department manager may need employee attendance and performance but does not inherently need home addresses, government identifiers or organization-wide payroll values.

Sample Supabase SQL foundation

The following is intentionally a starting architecture rather than the entire production schema.

```
-- UUID support is already readily available in modern Supabase/Postgres setups.

create type public.app_role as enum (
  'employee',
  'supervisor',
  'manager',
  'admin'
);

create table public.organizations (
  id uuid primary key default gen_random_uuid(),
  name text not null,
  timezone text not null default 'UTC',
  settings jsonb not null default '{} '::jsonb,
  created_at timestamptz not null default now()
);

create table public.employees (
  id uuid primary key default gen_random_uuid(),
  organization_id uuid not null
  references public.organizations(id) on delete restrict,
```

```

user_id uuid unique
    references auth.users(id) on delete set null,

employee_number text not null,
external_payroll_id text,
first_name text not null,
last_name text not null,
work_email text,
status text not null
    check (status in ('prehire', 'active', 'leave', 'terminated')),

hire_date date,
termination_date date,
created_at timestamptz not null default now(),
updated_at timestamptz not null default now(),

unique (organization_id, employee_number)
);

create index employees_org_idx
    on public.employees(organization_id);

create index employees_user_idx
    on public.employees(user_id);

create table public.role_assignments (
    id uuid primary key default gen_random_uuid(),
    user_id uuid not null references auth.users(id) on delete cascade,
    organization_id uuid not null
        references public.organizations(id) on delete cascade,

    role public.app_role not null,

    -- organization / org_unit / employee etc.
    scope_type text not null default 'organization',
    scope_id uuid,

    valid_from timestamptz not null default now(),
    valid_until timestamptz,

    created_at timestamptz not null default now()
);

create index role_assignments_lookup_idx
    on public.role_assignments(user_id, organization_id, role);

alter table public.organizations enable row level security;

```

```
alter table public.employees enable row level security;
alter table public.role_assignments enable row level security;
```

Tables created through SQL need RLS explicitly enabled; Supabase recommends RLS on exposed tables and notes that grants and policies should both be configured according to least privilege. ⁷

Use a private schema for security helper functions:

```
create schema if not exists private;

create or replace function private.current_employee_id()
returns uuid
language sql
stable
security definer
set search_path = ''
as $$
    select e.id
    from public.employees e
    where e.user_id = (select auth.uid())
    limit 1;
$$;

create or replace function private.has_org_role(
    p_org_id uuid,
    p_roles public.app_role[]
)
returns boolean
language sql
stable
security definer
set search_path = ''
as $$
    select exists (
        select 1
        from public.role_assignments ra
        where ra.user_id = (select auth.uid())
            and ra.organization_id = p_org_id
            and ra.role = any(p_roles)
            and ra.valid_from <= now()
            and (ra.valid_until is null or ra.valid_until > now())
    );
$$;
```

Supabase recommends careful handling of `security definer` functions, including setting `search_path`; Supabase's testing guidance also demonstrates keeping security-definer RLS helpers in a private schema rather than an exposed API schema. ³¹

Example employee self-access:

```
create policy "employee can read self"
on public.employees
for select
to authenticated
using (
    user_id = (select auth.uid())
);
```

Example organization Admin visibility:

```
create policy "admin can read organization employees"
on public.employees
for select
to authenticated
using (
    private.has_org_role(
        organization_id,
        array['admin']::public.app_role
    )
);
```

For supervisors/managers, use organizational assignment/scope information instead of blindly granting every Supervisor the entire organization's data.

At scale, create a precomputed scope table such as:

```
management_scope
-----
organization_id
actor_user_id
employee_id
scope_reason
```

Then supervisor/manager RLS becomes a fast indexed `EXISTS` check rather than repeatedly traversing a deep organization tree.

Columns referenced frequently in RLS policies should be indexed. Supabase's RLS performance guidance specifically recommends indexing columns used by policies and recommends wrapping stable auth helpers such as `auth.uid()` in `SELECT` in appropriate policies to reduce repeated evaluation. ³²

Attendance SQL and protected clock operations

```
create table public.attendance_sessions (  
  id uuid primary key default gen_random_uuid(),  
  organization_id uuid not null  
    references public.organizations(id),  
  employee_id uuid not null  
    references public.employees(id),  
  
  work_date date not null,  
  clock_in_at timestamptz not null,  
  clock_out_at timestamptz,  
  status text not null default 'open'  
    check (status in ('open', 'closed', 'corrected')),  
  
  created_at timestamptz not null default now()  
);  
  
create unique index attendance_one_open_session  
on public.attendance_sessions(employee_id)  
where clock_out_at is null;  
  
create index attendance_employee_date_idx  
on public.attendance_sessions(employee_id, work_date desc);  
  
alter table public.attendance_sessions enable row level security;  
  
create policy "employee reads own attendance"  
on public.attendance_sessions  
for select  
to authenticated  
using (  
  employee_id = private.current_employee_id()  
);
```

Do not provide employees unrestricted direct `UPDATE` permission to `clock_in_at` and `clock_out_at`. Perform state transitions through carefully permissioned database functions/RPCs.

Example:

```

create or replace function public.clock_in()
returns public.attendance_sessions
language plpgsql
security invoker
set search_path = ''
as $$
declare
    v_employee public.employees;
    v_session public.attendance_sessions;
begin
    select e.*
    into v_employee
    from public.employees e
    where e.user_id = (select auth.uid())
        and e.status = 'active';

    if v_employee.id is null then
        raise exception 'No active employee record';
    end if;

    if exists (
        select 1
        from public.attendance_sessions s
        where s.employee_id = v_employee.id
            and s.clock_out_at is null
    ) then
        raise exception 'Employee already clocked in';
    end if;

    insert into public.attendance_sessions (
        organization_id,
        employee_id,
        work_date,
        clock_in_at
    )
    values (
        v_employee.organization_id,
        v_employee.id,
        (now() at time zone 'UTC')::date,
        now()
    )
    returning * into v_session;

    return v_session;
end;
$$;

```

Function privileges should be explicitly controlled because PostgreSQL functions may otherwise receive broader default `EXECUTE` privileges than intended. Supabase recommends revoking unnecessary function execution grants. ³³

Storage RLS

Store metadata in `documents`, while objects follow a predictable organization/employee path.

Conceptually:

```
create policy "employees upload own HR documents"
on storage.objects
for insert
to authenticated
with check (
  bucket_id = 'employee-documents'
  and (storage.foldername(name))[1] = (
    select organization_id::text
    from public.employees
    where user_id = auth.uid()
  )
  and (storage.foldername(name))[2] = (
    select id::text
    from public.employees
    where user_id = auth.uid()
  )
);
```

Supabase Storage denies uploads without appropriate Storage RLS policies, and private buckets use authorization/RLS to protect reads as well. ³⁴

Keep every HR document bucket private. Public buckets intentionally make asset retrieval publicly accessible to anyone possessing the asset URL, which is not appropriate for employee records. ³⁵

Workflow automation, Excel payroll import and external integrations

Payroll import architecture

This is where your "no payroll engine" requirement should be strictly enforced.

The system should **never** determine:

```
hourly wage × hours
overtime rate
tax withholding
social-security contributions
pension deduction
statutory deductions
net pay
```

Those values arrive from the employer's payroll process.

The HR application becomes a **payroll-results repository and reconciliation layer**.

Recommended tables:

```
payroll_import_batches
-----
id
organization_id
pay_period_start
pay_period_end
pay_date
original_file_path
original_file_name
file_hash
status
uploaded_by
uploaded_at
approved_by
approved_at
supersedes_batch_id
total_rows
matched_rows
unmatched_rows
error_rows

payroll_import_rows
-----
id
batch_id
row_number
employee_id
external_employee_id
employee_number
employee_email
```



```
employee_name_raw
gross_pay
net_pay
currency
earnings_json
deductions_json
taxes_json
employer_values_json
raw_row_json
mapping_status
validation_status
error_message
```

```
payroll_column_maps
-----
id
organization_id
name
mapping_json
created_by
```

You do not need to force every payroll provider into the same spreadsheet layout.

For example, Employer A may upload:

```
Employee No | Gross | Tax | Pension | Net
```

while Employer B uploads:

```
Worker ID | Employee | Basic Pay | OT | PAYE | NIS | Net Pay
```

A column map translates them to your internal import representation.

Matching rules

Use deterministic identifiers first:

```
external_payroll_id
      ↓
employee_number
      ↓
work_email
```

↓
manual reconciliation

Avoid automatically approving name-only matches. Two employees can share a name, names change, names may contain spelling differences and spreadsheets regularly contain inconsistent formatting.

The import UI should display:

Matched automatically	147
Needs review	3
Invalid	1
Total	151

Nothing should commit as an approved batch until unresolved rows are deliberately handled according to employer policy.

Revision instead of overwrite

Suppose July payroll was imported incorrectly.

Do **not** run:

```
update payroll_records
set net_pay = ...
where employee_id = ...
```

and destroy the previous value.

Instead:

```
July Payroll Batch v1
    ↓ superseded by
July Payroll Batch v2
```

The latest approved revision becomes the visible/current result, while both versions remain in the audit history.

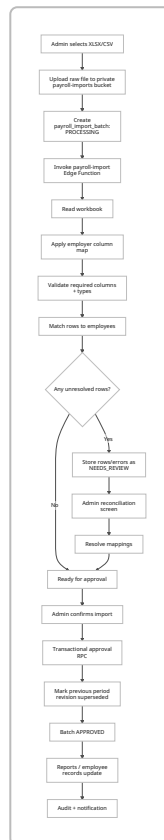
A reporting view might use:

```
create view public.current_payroll_records
with (security_invoker = true)
```

```
as
select ...
```

Using `security_invoker=true` on exposed views is important because Supabase documents that ordinary creator-owned PostgreSQL views can otherwise bypass underlying RLS. ²²

Excel processing flow



I recommend uploading the workbook to private Storage first rather than sending the original file only as a large direct function body. Supabase Storage supports normal file uploads, and recommends resumable TUS uploads when files are large or unreliable networks are a concern. ³⁶

The Edge Function can then download the file, parse it and insert staged rows.

Supabase Edge Functions now support npm modules, so a compatible XLSX parser can be included in the function dependency configuration. Supabase recommends per-function `deno.json` dependency management and supports npm imports. ³⁷

However, spreadsheet processing must be benchmarked. Hosted Edge Functions have runtime limits, including a 256 MB memory limit and CPU/wall-clock constraints. Very large workbooks or computationally heavy processing may require an external worker rather than attempting to force the job through an Edge Function. ³⁸

For larger asynchronous imports, use:

Upload → create batch → queue job → worker processes → database status →
Realtime UI

Supabase Queues provides durable message storage and controlled processing/retries, which makes it useful for notification and import orchestration. ¹³

Suggested minimal Edge Functions / RPC surface

Most CRUD does not need custom HTTP endpoints. Keep your custom backend surface small.

Function/RPC	Type	Purpose
<code>invite-employee</code>	Edge Function	Privileged Auth invitation and employee/user linking
<code>clock_in()</code>	PostgreSQL RPC	Atomic attendance state transition
<code>clock_out()</code>	PostgreSQL RPC	Atomic attendance state transition
<code>request_attendance_adjustment()</code>	RPC	Submit clock correction
<code>decide_attendance_adjustment()</code>	RPC	Supervisor/Admin correction
<code>submit_leave()</code>	RPC	Validate and submit leave
<code>decide_leave_request()</code>	RPC	Transactional approval/rejection
<code>start-onboarding()</code>	RPC/Edge	Instantiate template
<code>complete-onboarding-task()</code>	RPC	Validate step completion
<code>launch-appraisal-cycle()</code>	RPC	Create appraisal instances
<code>submit-appraisal()</code>	RPC	Lock appropriate response stage
<code>payroll-import</code>	Edge Function	Parse/validate uploaded workbook
<code>approve_payroll_import()</code>	RPC	Atomically activate reconciled batch
<code>send-notifications</code>	Edge Function	External email/SMS/push delivery
<code>signature-webhook</code>	Edge Function	Receive external e-signature status
<code>export-report</code>	Edge Function	Sensitive or large HR exports

Database functions are appropriate for data-intensive operations that belong close to PostgreSQL; Edge Functions are appropriate for server-side integration/network orchestration. Supabase documents both patterns. ³⁹

Scheduled jobs

Schedule	Job
Every few minutes	Check notification queue / retry failed deliveries
Daily	Missing clock-out detection
Daily	Upcoming onboarding deadlines
Daily	Appraisal deadline reminders
Daily	Certification/document expiry reminders
Daily	Scheduled employee status transitions if enabled
Monthly	Leave accrual
Yearly/policy date	Leave grant/carryover/expiry
Appraisal-cycle opening time	Generate appraisal instances
Retention schedule	Archive or purge eligible records according to employer policy

Supabase Cron can run SQL, call database procedures or send HTTP requests to Edge Functions. Supabase currently recommends limiting job concurrency and duration; its Cron documentation notes a recommendation of no more than eight concurrent jobs and jobs under ten minutes. ⁴⁰

For complicated workflows, Cron should enqueue work rather than perform all external network operations inside one giant scheduled SQL transaction.

External API boundary

Requirement	Supabase alone?	Recommendation
HR database	Yes	PostgreSQL
Login	Yes	Auth
Passwordless login	Yes	Auth
MFA	Yes	Auth
SAML SSO	Yes, with organization's IdP	Supabase SAML SSO
Attendance	Yes	PostgreSQL + RPC + Realtime
Leave	Yes	PostgreSQL + Cron
Appraisals	Yes	PostgreSQL + Cron
Onboarding	Yes	PostgreSQL + Storage + Cron
In-app notifications	Yes	PostgreSQL + Realtime

Requirement	Supabase alone?	Recommendation
Production email	External delivery infrastructure recommended	Edge Function + provider
SMS	External provider	Edge Function
Mobile push	External APNs/FCM-type service	Edge Function
Basic acknowledgements	Yes	PostgreSQL + audit metadata
High-assurance/legal e-signature	External e-signature provider recommended	Edge Function/webhook
Excel payroll upload	Yes for ordinary workloads	Storage + Edge Function + PostgreSQL
Extremely large spreadsheet ETL	Possibly not	External worker if Edge limits become restrictive
Payroll calculation	No — deliberately excluded	Employer's external payroll solution
Automated payroll-provider sync	External API required	Provider integration
Document malware scanning	External scanner recommended	Upload → queue → scanner → approved document

Security, auditability, reporting, backup and testing

Security model

HR applications contain unusually sensitive information, so authorization should be designed as a database feature rather than frontend logic.

Supabase's Data API uses PostgreSQL grants and RLS as two separate control layers. A table can have RLS policies and still have unnecessarily broad table/function grants, so configure both. ⁴¹

Recommended security rules:

Every exposed HR table gets RLS.

Supabase warns that tables in exposed schemas without RLS can be accessible through the Data API. ⁴²

No `service_role` or secret key in browser/mobile code.

Supabase's service/secret credentials bypass RLS and are explicitly intended only for secure server environments. ⁴³

Separate highly sensitive HR data.

For example:

employees	normal HR profile
employee_private	personal PII
employee_medical_documents	extremely restricted
payroll_import_rows	payroll permission
audit_events	audit permission

That approach is usually easier to reason about than one gigantic row where every query risks exposing a sensitive column. Supabase additionally supports column-level privileges when needed. ⁴⁴

Use private Storage buckets.

Private-bucket reads are controlled through authorization/RLS, while public buckets make object retrieval public. ¹⁰

Require MFA for powerful administrative actions.

Supabase Auth provides MFA and RLS can test the Authenticator Assurance Level. ⁴⁵

Index authorization columns.

Indexes should normally exist on:

```
organization_id
user_id
employee_id
manager_employee_id
supervisor_employee_id
org_unit_id
role_assignments(user_id, organization_id)
management_scope(actor_user_id, employee_id)
```

Supabase's RLS performance guidance explicitly recommends indexes for columns used inside RLS predicates. ⁴⁶

Audit architecture

Use three different audit layers rather than treating them as interchangeable.

Application audit: your `audit_events` table.

Record actions such as:

```
EMPLOYEE_CREATED
EMPLOYEE_UPDATED
ROLE_ASSIGNED
ROLE_REMOVED
CLOCK_CORRECTED
LEAVE_APPROVED
LEAVE_REJECTED
APPRAISAL_SUBMITTED
DOCUMENT_DOWNLOADED
PAYROLL_FILE_UPLOADED
PAYROLL_IMPORT_APPROVED
REPORT_EXPORTED
```

For sensitive mutations store:

```
{
  "actor_user_id": "...",
  "entity_type": "leave_request",
  "entity_id": "...",
  "action": "APPROVED",
  "old_data": {"status": "submitted"},
  "new_data": {"status": "approved"},
  "created_at": "..."
}
```

Avoid placing credentials, raw passwords or unnecessary highly sensitive document contents into audit payloads.

Supabase Auth Audit Logs: authentication activity. Supabase automatically records authentication events including signup/login, password/reset-related operations, verification, refresh and logout activity. ¹⁵

Database/platform audit: optional additional controls. Supabase supports PGAudit for selective PostgreSQL auditing, while Platform Audit Logs are available on Team/Enterprise for project/platform administrative activity. ⁴⁷

Reporting and analytics

Initial reports can stay entirely inside PostgreSQL.

Recommended dashboards:

People: active headcount, hires, departures, department distribution, location distribution and employment status.

Attendance: clocked-in employees, attendance rate, late arrivals, missing clock-outs, hours by employee/team and manual adjustments.

Leave: balance utilization, approved leave, pending requests, absence calendar, leave by type and departmental trends.

Onboarding: percentage complete, overdue tasks, average onboarding completion time and incomplete required documents.

Performance: appraisal completion, overdue reviews, rating distributions where organizational policy permits, checkpoint completion and goals.

Training/compliance: expiring certifications and required courses.

Payroll-import: batch status, mismatched employee rows, imported gross/net values, corrections and reconciliation history. It should remain reporting on externally calculated data, not calculating payroll.

Use normal PostgreSQL views for many dashboards:

```
create view public.employee_headcount_v
with (security_invoker = true)
as
select
  organization_id,
  count(*) filter (where status = 'active') as active_count,
  count(*) filter (where status = 'prehire') as prehire_count
from public.employees
group by organization_id;
```

The `security_invoker` setting is important for exposed reporting views because Supabase documents that standard PostgreSQL views created by privileged owners can bypass RLS. ⁴⁸

For expensive analytics, do not expose an unrestricted materialized view containing every organization's data through the API. Supabase's Security Advisor explicitly checks for potentially unsafe materialized-view/API exposure. ⁴⁹

Backup and disaster recovery

Supabase manages database backups and provides Point-in-Time Recovery on eligible paid configurations; PITR allows restoration to a chosen point with fine time granularity. ⁵⁰

For production HR data, I would recommend PITR rather than relying solely on periodic backups because losing even a day's employee approvals, attendance and personnel changes could be operationally significant.

There is an important limitation: **Supabase database backups do not back up Storage objects**. Supabase explicitly states that database backups cover the database itself but not files stored through the Storage API. ⁵¹

Also, Supabase Storage's S3 compatibility currently does not provide S3 object versioning, and deleted objects are not restorable through Storage versioning. ⁵²

Therefore implement two concepts:

Database history:
Postgres backups + PITR

Document history:
document_versions table + independent Storage backup/replication process

For strong disaster recovery of HR files, periodically replicate critical Storage objects to another backup destination—potentially another controlled Supabase project or an independent object-storage system. That external copy is one of the few areas where a strict "one Supabase project is everything" architecture would create unnecessary recovery risk.

Test restores. A backup strategy that has never been restored is not a complete disaster-recovery process.

Supabase supports logical backup/restore workflows with PostgreSQL tools and also provides restore/clone mechanisms for supported projects. ⁵³

Testing strategy

Do not test only frontend screens. Most of the security-critical behavior lives in PostgreSQL.

Supabase provides local development through the CLI and supports schema migrations, local environments and generated project configuration. ⁵⁴

The test pyramid should include:

Testing layer	Required tests
Schema	Constraints, foreign keys, uniqueness, valid status transitions
RLS	Employee cannot read coworker's private data; Supervisor cannot read outside scope; Manager only sees subtree; Admin permissions behave correctly
RPC	Double clock-in, double clock-out, leave over-allocation, concurrent approval, unauthorized approval
Workflow	Template versioning, onboarding dependencies, appraisal stage transitions

Testing layer	Required tests
Payroll import	Valid workbook, reordered columns, missing columns, duplicate employees, unknown employees, malformed numeric values, correction batch
Edge Functions	Authentication, role verification, file-validation errors, external-provider timeout/retry
Storage	Cross-employee access denial, wrong bucket, signed URL expiry/access
Realtime	Unauthorized users cannot subscribe to restricted channels
Cron	Idempotency, duplicate-run protection, timezone boundaries
Audit	Every sensitive action creates expected audit event
Recovery	Database restore and document recovery drills

Supabase's official testing guidance supports PostgreSQL unit/security testing with pgTAP, including tests specifically for RLS policies, and the CLI provides `supabase test db` plus Edge Function testing. ⁵⁵

For example, create identities representing:

```
Alice – Employee – Department A
Bob – Supervisor – Department A
Carol – Manager – Division A
David – Employee – Department B
Erin – Admin
```

Then deliberately test forbidden actions:

```
Alice queries David's leave request      → 0 rows / forbidden
Bob queries Department B payroll         → forbidden
Carol modifies Erin's role               → forbidden
Alice changes clock_in_at directly       → forbidden
Bob downloads Alice's medical attachment → forbidden unless explicitly
permitted
Erin imports payroll with import permission → allowed
```

This is especially important because an HR security failure is usually not "the user could not log in"; it is "the logged-in user could access one row they were never supposed to see."

Supabase also supplies Security and Performance Advisors that detect issues such as missing RLS, problematic security-definer exposure, unindexed relationships and exposed sensitive structures. Run them as part of release preparation. ⁵⁶

Recommended implementation blueprint

The system should be developed as a sequence of coherent HR capabilities rather than trying to launch every module simultaneously.

Foundation: organizations, employees, org units, positions, assignments, Auth, role assignments, permissions, audit events, Storage structure and RLS. This needs to be correct first because every subsequent feature inherits its employee identity and authorization model.

Employee and attendance: build Employee/Supervisor/Admin portal shells, employee directory/profile, attendance policies, schedules, clock-in/out, break support, adjustment workflow and live team status.

Leave: implement configurable leave types, policies, ledger balances, employee submission, approval chain, calendars, attachments, Cron-based accrual and reminders.

Workflow engine: create reusable concepts for templates, template versions, instances, tasks, dependencies, assignees, due dates and status changes. Use that foundation for onboarding first, then offboarding and potentially other HR processes.

Performance: add appraisal templates, questions, cycles, checkpoints, reviewers, self-assessment, supervisor/manager reviews, acknowledgement and scheduled generation.

Documents: private file management, categories, expiration dates, policy acknowledgements and optional e-signature provider integration.

Payroll import: build column mappings, secure XLSX Storage, parsing, staging, employee reconciliation, validation, approval, revisioning and restricted payroll reports. Keep external payroll values immutable and never introduce wage/tax calculation logic.

Analytics and enterprise capabilities: reporting dashboards, exports, SSO, more sophisticated role customization, notifications, training/certification tracking and equipment/offboarding.

A sensible Supabase project structure would resemble:

```
supabase/  
├─ migrations/  
│   └─ organizations.sql  
│   └─ employees.sql  
│   └─ authorization.sql  
│   └─ attendance.sql  
│   └─ leave.sql  
│   └─ workflow_engine.sql  
│   └─ onboarding.sql  
│   └─ appraisal.sql  
│   └─ documents.sql
```

```

|   ├── payroll_import.sql
|   ├── notifications.sql
|   ├── audit.sql
|   └── reporting.sql
|
|   ├── functions/
|   |   ├── invite-employee/
|   |   ├── payroll-import/
|   |   ├── send-notifications/
|   |   ├── signature-webhook/
|   |   └── export-report/
|   |
|   ├── tests/
|   |   ├── database/
|   |   |   ├── employee_rls_test.sql
|   |   |   ├── attendance_rls_test.sql
|   |   |   ├── leave_rls_test.sql
|   |   |   ├── appraisal_rls_test.sql
|   |   |   ├── payroll_rls_test.sql
|   |   |   └── workflow_test.sql
|   |
|   ├── seed.sql
|   └── config.toml

```

Supabase's recommended local-development workflow is migration-oriented and designed so the `supabase` directory can reproduce the project locally or against a fresh environment; seed files are applied after migrations and are intended for data insertion rather than schema definitions. ⁵⁷

The most important architectural principles for the final product are:

1. **PostgreSQL is the HR system of record.**
2. **RLS, not frontend menus, controls authorization.**
3. **Roles are scoped; Supervisor and Manager do not automatically gain organization-wide HR access.**
4. **Sensitive HR categories are separated and permissioned independently.**
5. **Attendance timestamps come from trusted database operations rather than editable browser fields.**
6. **Realtime enhances the user experience but is not the authoritative attendance ledger.**
7. **Onboarding and appraisal processes are employer-configurable, versioned workflows rather than hard-coded forms.**
8. **Leave types and policies are employer data, not application constants.**
9. **Documents stay in private Storage and are represented by metadata/version records in PostgreSQL.**
10. **Payroll is imported as immutable externally calculated results; this system is deliberately not a payroll engine.**
11. **Every important change is auditable.**

12. **Cron generates recurring HR work; Queues handle reliable asynchronous processing.**
13. **External APIs are introduced only where Supabase does not itself provide the business service —general outbound communications, specialized e-signature, payroll-provider integration, malware scanning or workloads that exceed Edge Function constraints.**
14. **Database and RLS tests are first-class product code, because a correct HR portal is fundamentally an authorization system around highly sensitive employee data.**

With that architecture, the four portal views become different perspectives over the same coherent HR data model rather than independent systems: the **Employee portal** handles self-service, the **Supervisor portal** handles direct-report operations, the **Manager portal** handles broader organizational accountability, and the **Admin portal** configures and governs the HR platform. Supabase then serves as the security and data layer binding attendance, leave, onboarding, performance, documents, notifications, reporting, audit and external-payroll import into one integrated HR system.

1 5 51 Database | Supabase Docs

https://supabase.com/docs/guides/database/overview?utm_source=chatgpt.com

2 Custom Claims & Role-based Access Control (RBAC)

https://supabase.com/docs/guides/api/custom-claims-and-role-based-access-control-rbac?utm_source=chatgpt.com

3 Securing your data | Supabase Docs

https://supabase.com/docs/guides/database/secure-data?utm_source=chatgpt.com

4 User Management | Supabase Docs

https://supabase.com/docs/guides/auth/managing-user-data?utm_source=chatgpt.com

6 Auth | Supabase Docs

https://supabase.com/docs/guides/auth?utm_source=chatgpt.com

7 21 22 30 42 48 Row Level Security | Supabase Docs

https://supabase.com/docs/guides/database/postgres/row-level-security?utm_source=chatgpt.com

8 Custom Access Token Hook | Supabase Docs

https://supabase.com/docs/guides/auth/auth-hooks/custom-access-token-hook?utm_source=chatgpt.com

9 24 Realtime Authorization | Supabase Docs

https://supabase.com/docs/guides/realtime/authorization?utm_source=chatgpt.com

10 27 35 Storage Buckets | Supabase Docs

https://supabase.com/docs/guides/storage/buckets/fundamentals?utm_source=chatgpt.com

11 29 Edge Functions | Supabase Docs

https://supabase.com/docs/guides/functions?utm_source=chatgpt.com

12 25 40 Cron | Supabase Docs

https://supabase.com/docs/guides/cron?utm_source=chatgpt.com

13 Supabase Queues | Supabase Docs

https://supabase.com/docs/guides/queues?utm_source=chatgpt.com

14 Vault | Supabase Docs

https://supabase.com/docs/guides/database/vault?utm_source=chatgpt.com

15 Auth Audit Logs | Supabase Docs

https://supabase.com/docs/guides/auth/audit-logs?utm_source=chatgpt.com

16 Platform Audit Logs | Supabase Docs

https://supabase.com/docs/guides/security/platform-audit-logs?utm_source=chatgpt.com

17 43 Understanding API keys | Supabase Docs

https://supabase.com/docs/guides/getting-started/api-keys?utm_source=chatgpt.com

18 Passwordless email logins | Supabase Docs

https://supabase.com/docs/guides/auth/auth-email-passwordless?utm_source=chatgpt.com

19 45 Multi-Factor Authentication | Supabase Docs

https://supabase.com/docs/guides/auth/auth-mfa?utm_source=chatgpt.com

20 Single Sign-On with SAML 2.0 for Projects

https://supabase.com/docs/guides/auth/enterprise-sso/auth-sso-saml?utm_source=chatgpt.com

23 Presence | Supabase Docs

https://supabase.com/docs/guides/realtime/presence?utm_source=chatgpt.com

26 Scheduling Edge Functions | Supabase Docs

https://supabase.com/docs/guides/functions/schedule-functions?utm_source=chatgpt.com

28 Broadcast | Supabase Docs

https://supabase.com/docs/guides/realtime/broadcast?utm_source=chatgpt.com

31 33 39 Database Functions | Supabase Docs

https://supabase.com/docs/guides/database/functions?utm_source=chatgpt.com

32 46 RLS Performance and Best Practices

https://supabase.com/docs/guides/troubleshooting/rls-performance-and-best-practices-Z5Jjwv?utm_source=chatgpt.com

34 Storage Access Control | Supabase Docs

https://supabase.com/docs/guides/storage/security/access-control?utm_source=chatgpt.com

36 Standard Uploads | Supabase Docs

https://supabase.com/docs/guides/storage/uploads/standard-uploads?utm_source=chatgpt.com

37 Managing dependencies | Supabase Docs

https://supabase.com/docs/guides/functions/dependencies?utm_source=chatgpt.com

38 Limits | Supabase Docs

https://supabase.com/docs/guides/functions/limits?utm_source=chatgpt.com

41 Securing your API | Supabase Docs

https://supabase.com/docs/guides/api/securing-your-api?utm_source=chatgpt.com

44 Column Level Security | Supabase Docs

https://supabase.com/docs/guides/database/postgres/column-level-security?utm_source=chatgpt.com

47 PGAudit: Postgres Auditing | Supabase Docs

https://supabase.com/docs/guides/database/extensions/pgaudit?utm_source=chatgpt.com

49 56 Performance and Security Advisors | Supabase Docs

https://supabase.com/docs/guides/database/database-advisors?utm_source=chatgpt.com

50 Database Backups | Supabase Docs

https://supabase.com/docs/guides/platform/backups?utm_source=chatgpt.com

52 S3 Compatibility | Supabase Docs

https://supabase.com/docs/guides/storage/s3/compatibility?utm_source=chatgpt.com

53 Backup and Restore using the CLI | Supabase Docs

https://supabase.com/docs/guides/platform/migrating-within-supabase/backup-restore?utm_source=chatgpt.com

54 57 Local development workflow - Docs

https://supabase.com/docs/guides/local-development/cli-workflows?utm_source=chatgpt.com

55 Testing Overview | Supabase Docs

https://supabase.com/docs/guides/local-development/testing/overview?utm_source=chatgpt.com